```
function [phase,ens_coh,flags] =
ps_region_growing(phase,ens_coh,flags,index_list,Btemp,Bdop,std_param,h2ph,final
_model,final_althyp_index)

% Input:    -
%
% Output:   -
%
% ----------------------------------------------------------------------
% File............: ps_region_growing.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Authors.........: Freek van Leijen
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ----------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
%



global fig althyp_index

ens_coh_thres = 0.9;
[Nx,Ny,Nifgs] = size(phase);
phase_window_orig = repmat(NaN,[5 5 Nifgs]);
ens_coh_window_orig = repmat(NaN,5,5);
flags_window_orig = repmat(NaN,5,5);

althyp_index = final_althyp_index;
[B1Qy2,par_index,covar_index,defo_index] =
ps_model_definitions('model',final_model,Nifgs,h2ph,Btemp,Bdop,std_param);
B1 = B1Qy2(1:Nifgs,:);
N = B1'*B1;
R = chol(N);
rhs = R\(R'\B1');

while ~isempty(index_list)&ens_coh_thres>0.4
  while ~isempty(index_list)
    [dummy,index] = max(index_list(:,3));
    x = index_list(index,1);
    y = index_list(index,2);

    for v = 1:5

      phase_window = phase_window_orig;
      ens_coh_window = ens_coh_window_orig;
      flags_window = flags_window_orig;

      if v==1
        a = x;
        b = y;
      elseif v==2
        a = x-1;
        b = y;
      elseif v==3
        a = x+1;
        b = y;
```

```
      elseif v==4
        a = x;
        b = y-1;
      elseif v==5
        a = x;
        b = y+1;
      end

      if (a>0 & a<=Nx & b>0 & b<=Ny & flags(a,b)~=1) %not already unwrapped

        a1 = max(1,a-2);
        a2 = min(Nx,a+2);
        b1 = max(1,b-2);
        b2 = min(Ny,b+2);
        phase_window(a1-a+3:a2-a+3,b1-b+3:b2-b+3,:) = phase(a1:a2,b1:b2,:);
        ens_coh_window(a1-a+3:a2-a+3,b1-b+3:b2-b+3) = ens_coh(a1:a2,b1:b2);
        flags_window(a1-a+3:a2-a+3,b1-b+3:b2-b+3) = flags(a1:a2,b1:b2);

        [phase_unw,ens_coh_local,out_flag] = predict_phase(phase_window,...
           flags_window,ens_coh_window,rhs,B1,ens_coh_thres,Nifgs);

        if out_flag==0
          flags(a,b) = 0; %no unwrapped neighbours
        elseif out_flag==1
          flags(a,b) = 1; %unwrapped flag
          phase(a,b,:) = phase_unw;
          ens_coh(a,b) = ens_coh_local;
          index_list = [index_list;[a b ens_coh_local]];
        elseif out_flag==2
          flags(a,b) = 2; %postponed flag
          ens_coh(a,b) = ens_coh_local;
        end

      end
    end
    index_list(index,:) = [];
    %figure(fig);
    %imagesc(flags);colorbar
    %figure(fig+1);
    %imagesc(ens_coh);colorbar
    %display('pause')
    %pause
  end
  index = find(flags(:)==2); %find postponed pixels
  [x_index,y_index] = ind2sub([Nx Ny],index);
  index_list = [x_index y_index ens_coh(index)];
  %this ens_coh value is not very reliable, but a value is needed
  %to select the next pixel
  ens_coh_thres = ens_coh_thres-0.1; % lower threshold
end


function [phase_unw,ens_coh_local,out_flag] =
predict_phase(phase_window,flags_window,ens_coh_window,rhs,B1,ens_coh_thres,Nifg
s);

pred = repmat(NaN,Nifgs,8);
weight = repmat(NaN,8,1);
ens_coh = repmat(NaN,8,1);

dx = [2 1;2 1;3 3;4 5;4 5;4 5;3 3;2 1];
dy = [3 3;4 5;4 5;4 5;3 3;2 1;2 1;2 1];

for v = 1:8
```

```
  flags = flags_window(dx(v,1),dy(v,1));
  if flags==1
    pred(:,v) = phase_window(dx(v,1),dy(v,1),:);
    weight(v) = 0.5;
    ens_coh(v) = ens_coh_window(dx(v,1),dy(v,1));
    flags = flags_window(dx(v,2),dy(v,2));
    if flags==1
      pred(:,v) = 2*phase_window(dx(v,1),dy(v,1),:)-
phase_window(dx(v,2),dy(v,2),:);
      weight(v) = 1;
    end
  end
end

index = find(~isnan(weight));
if isempty(index)
  out_flag = 0; %no unwrapped neighbours
  phase_unw = [];
  ens_coh_local = [];
else
  [dummy,ens_coh_index] = max(ens_coh);
  pred = pred(:,index)*weight(index)/sum(weight(index));

  phase_wrap = reshape(phase_window(3,3,:),Nifgs,1);
  ambi = round((pred-phase_wrap)/(2*pi));

  phase_unw = 2*pi*ambi+phase_wrap;
  dphase_unw = phase_unw-
reshape(phase_window(dx(ens_coh_index,1),dy(ens_coh_index,1),:),Nifgs,1);
  xhat = rhs*dphase_unw;
  yhat = B1*xhat;
  ehat = dphase_unw-yhat;
  ens_coh_local = abs((1/Nifgs)*sum(exp(i*ehat')));
  if ens_coh_local>ens_coh_thres
    out_flag = 1;
  else
    out_flag = 2;
  end
end
```